Linear regression

Định giá căn hộ từ diện tích và khoảng cách

Phase 2 · Bài 2.1

Hời thật không?

Một căn hộ **70 m²**, cách trung tâm **8 km**, rao bán **4,2 tỷ**.

“Giá này là hời lắm rồi em ạ.”

Trong tay có giá của **200 căn** đã bán quanh đó. Làm sao trả lời bằng số liệu?

TRỰC GIÁC

Căn rộng hơn thì đắt hơn. Xa trung tâm thì rẻ đi.

Linear regression là trực giác này được viết thành toán học.

Sau buổi này, các em có thể

- Hiểu mô hình tuyến tính giả định điều gì và đọc được ý nghĩa từng hệ số.
- Tìm nghiệm bằng hai cách: nghiệm đóng và gradient descent.
- Giải thích vì sao learning rate lớn thì loss nổ, và vì sao phải chuẩn hóa đặc trưng.
- Dùng scikit-learn đúng quy trình và tránh các bẫy cài đặt hay gặp.

01

Dữ liệu và mô hình

Mô hình tuyến tính

Mỗi đặc trưng đóng góp một lượng tỉ lệ với nó, rồi cộng lại:

$$\hat{y} = w_1 x_1 + \cdots + w_d x_d + b = \mathbf{w}^\top \mathbf{x} + b$$

w_j: dự đoán tăng bao nhiêu khi x_j tăng 1 đơn vị **và các đặc trưng khác giữ nguyên**.

Cho cả n mẫu cùng lúc:

$$\hat{\mathbf{y}} = X\mathbf{w} + b$$

Ký hiệu	Shape
X	`(n, d)`
$\mathbf{w}$	`(d,)`
$\hat{\mathbf{y}}$, $\mathbf{y}$	`(n,)`

Dữ liệu của buổi học

Tự sinh 200 căn theo một công thức biết trước, cộng nhiễu:

$$\text{giá} = 0.05 \cdot \text{diện tích} - 0.15 \cdot \text{khoảng cách} + 1$$

Biết trước đáp án thì kiểm tra được mô hình học đúng hay sai.

```
rng = np.random.default_rng(0)
n = 200
area = rng.uniform(30, 120, n)
dist = rng.uniform(1, 15, n)
y = 0.05 * area - 0.15 * dist + 1.0 \
    + rng.normal(0, 0.3, n)
X = np.column_stack([area, dist])
print(X.shape, y.shape)
```

KẾT QUẢ

```
(200, 2) (200,)
```

HỎI NHANH

`X` có shape `(200, 2)`, `w` có shape `(2,)`. Biểu thức `X @ w + b` có shape gì?

A `(200,)`

B `(2,)`

C `(200, 2)`

D `(200, 1)`

02

Hàm mất mát

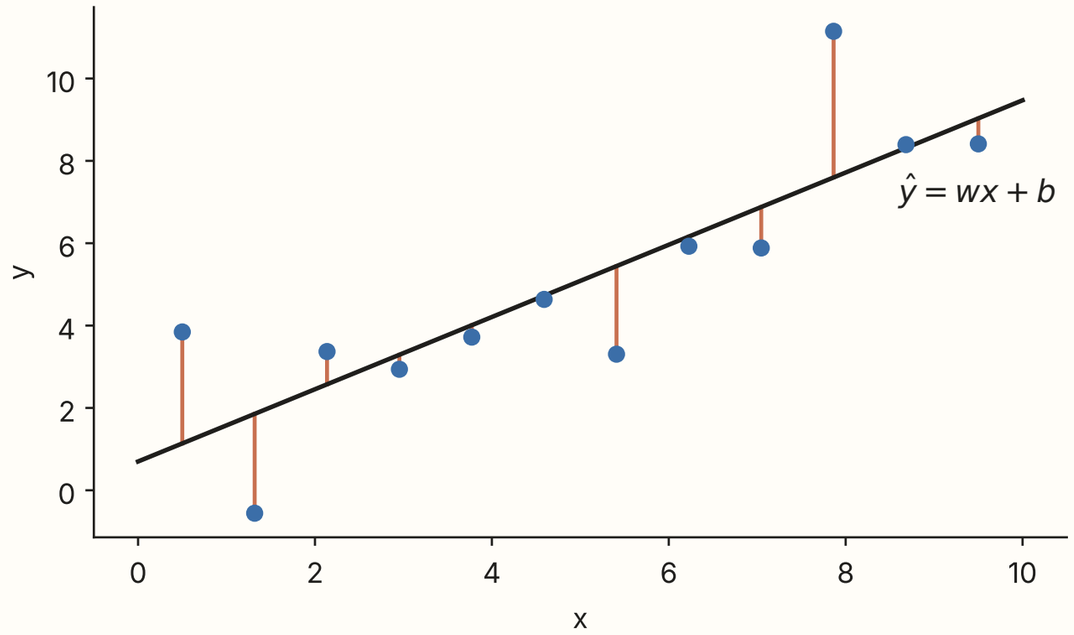
Mean squared error

$$J(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i)^2$$

Vì sao bình phương?

- Trơn, đạo hàm được ở mọi điểm.
- Phạt lỗi lớn nặng hơn: sai 10 bị phạt gấp 4 lần sai 5.
- Nhiễu phân phối chuẩn thì MSE là lựa chọn hợp lý nhất.

Nhược điểm: nhạy với outlier.



Phần dư của từng điểm. Least squares chọn đường làm tổng bình phương các đoạn đỏ nhỏ nhất.

Mốc so sánh

Mô hình đoán mọi căn bằng giá trung bình có

$$\text{MSE} = \text{Var}(y) = 2.34$$

Mô hình nào không thắng được mốc này thì chưa học được gì.

$$R^2 = 1 - \frac{\text{MSE}}{\text{Var}(y)}$$

$R^2 = 0$: ngang mốc · $R^2 = 1$: đoán trúng hết

03

Nghiệm đóng

Phương trình chuẩn

Thêm cột 1 vào X: $\tilde{X} = [X \;\; \mathbf{1}]$, gộp $\boldsymbol{\theta} = (\mathbf{w}, b)$, khi đó $\hat{\mathbf{y}} = \tilde{X}\boldsymbol{\theta}$.

Cực tiểu của J là nơi gradient bằng 0:

$$\nabla J(\boldsymbol{\theta}) = \frac{2}{n}\tilde{X}^\top(\tilde{X}\boldsymbol{\theta} - \mathbf{y}) = \mathbf{0} \quad \Longleftrightarrow \quad \tilde{X}^\top\tilde{X}\,\boldsymbol{\theta} = \tilde{X}^\top\mathbf{y}$$

Hệ $d+1$ phương trình, $d+1$ ẩn: giải bằng `np.linalg.solve`, không nghịch đảo ma trận.

Giải trên dữ liệu căn hộ

```
Xb = np.column_stack([X, np.ones(n)])
theta = np.linalg.solve(Xb.T @ Xb, Xb.T @ y)
w_cf, b_cf = theta[:2], theta[2]
print(w_cf.round(4), round(b_cf, 4))
print(mse(predict(X, w_cf, b_cf), y))
print(predict(np.array([70, 8]), w_cf, b_cf))
```

KẾT QUẢ

```
[ 0.0499 -0.1453] 0.9809
0.08807615180405967
3.309815169606732
```

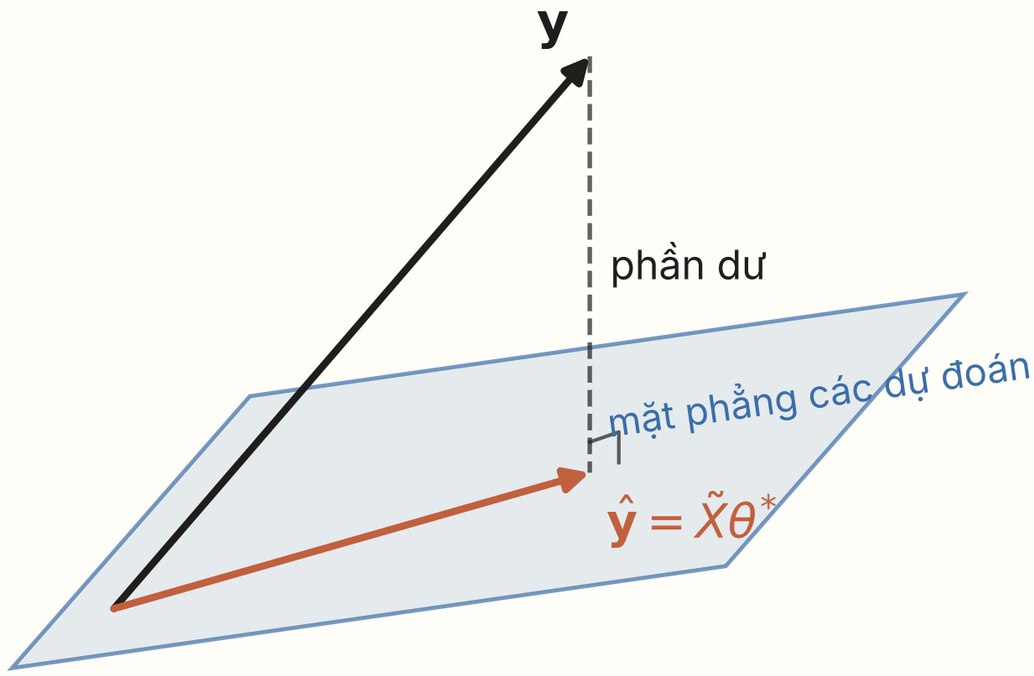
- Hệ số $\approx (0.05,\ -0.15,\ 1)$: khôi phục đúng công thức sinh dữ liệu.
- MSE $0.088 \approx 0.3^2$: phần còn lại là nhiễu, không học thêm được.
- Căn 70 m², 8 km: mô hình định giá **3,3 tỷ**. Rao 4,2 tỷ là cao hơn 0,9 tỷ.

Cách nhìn hình học

$\mathbf{y}$ là một vector trong không gian n chiều. Mọi dự đoán $\tilde{X}\boldsymbol{\theta}$ nằm trên mặt phẳng sinh bởi các cột của $\tilde{X}$.

Least squares chọn **hình chiếu vuông góc** của $\mathbf{y}$ lên mặt phẳng đó.

Phần dư vuông góc với mọi cột, kể cả cột số 1, nên tổng phần dư bằng 0.



Dự đoán là hình chiếu vuông góc của y; phần dư vuông góc với mặt phẳng các dự đoán.

04

Gradient descent

Đi xuống trong sương mù

Không thấy đáy, chỉ cảm nhận độ dốc dưới chân: bước một bước nhỏ về phía dốc xuống, rồi lặp lại.

Với $r_i = \hat{y}_i - y_i$:

$$\frac{\partial J}{\partial \mathbf{w}} = \frac{2}{n} X^\top \mathbf{r}, \qquad \frac{\partial J}{\partial b} = \frac{2}{n} \sum_i r_i$$

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \frac{\partial J}{\partial \mathbf{w}}, \qquad b \leftarrow b - \eta \frac{\partial J}{\partial b}$$

```
def grad(theta, X, y):
    r = predict(X, theta[:-1], theta[-1]) - y
    return np.append(2 / len(y) * X.T @ r,
                     2 * r.mean())


def gd(X, y, lr, epochs):
    theta = np.zeros(X.shape[1] + 1)
    history = []
    for _ in range(epochs):
        history.append(loss(theta, X, y))
        theta -= lr * grad(theta, X, y)
    return theta, history
```

HỎI NHANH

Với `r = y_hat - y`, dòng nào cập nhật bias đúng?

A `b -= lr * 2 * r.mean()`

B `b += lr * 2 * r.mean()`

C `b -= lr * 2 * r.sum()`

D `b -= lr * r`

Tính tay trên lớp

Ba điểm $(1, 2)$, $(2, 3)$, $(3, 5)$.

1. Tìm đường least squares $\hat{y} = wx + b$.
2. Từ $w = b = 0$, $\eta = 0.1$, làm một bước gradient descent.

Công thức cho một đặc trưng:

$$w = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}, \qquad b = \bar{y} - w\bar{x}$$

Lời giải

$\bar{x} = 2$, $\bar{y} = 10/3$, tử số $= 3$, mẫu số $= 2$:

$$w = 1.5, \qquad b = \tfrac{1}{3}, \qquad J = 0.056$$

Tại $w = b = 0$: $r = (-2, -3, -5)$,

$$\frac{\partial J}{\partial w} = -\frac{46}{3}, \qquad \frac{\partial J}{\partial b} = -\frac{20}{3}$$

Sau một bước: $w = 1.53$, $b = 0.67$, loss từ 12.67 xuống 0.22.

Kiểm tra gradient

Xấp xỉ đạo hàm chỉ bằng hàm loss, như viết code trâu để đối chiếu:

$$\frac{\partial J}{\partial \theta_j} \approx \frac{J(\boldsymbol{\theta} + h\mathbf{e}_j) - J(\boldsymbol{\theta} - h\mathbf{e}_j)}{2h}$$

Sai số $O(h^2)$, chọn $h \approx 10^{-5}$.

Phiên bản `grad`	Sai số tương đối
Đúng	$7 \cdot 10^{-13}$
Quên hệ số 2	0.333

Dưới 10^{-7}: yên tâm. Trên 10^{-2}: chắc chắn có lỗi.

05

Learning rate và chuẩn hóa

Thử ba learning rate

```
for lr in [1e-3, 2e-4, 1e-4]:
    theta_gd, hist = gd(X, y, lr, 1000)
    print(f"lr = {lr:g}: loss sau 1000 vòng = {hist[-1]:.4g}")
```

KẾT QUẢ

```
lr = 0.001: loss sau 1000 vòng = nan
lr = 0.0002: loss sau 1000 vòng = inf
lr = 0.0001: loss sau 1000 vòng = 0.1541
```

- Hai giá trị đầu: loss nổ tung.
- 10^{-4}: hội tụ nhưng rất chậm; sau 1000 vòng, b mới đi được 0.01 trên đường tới 0.98.

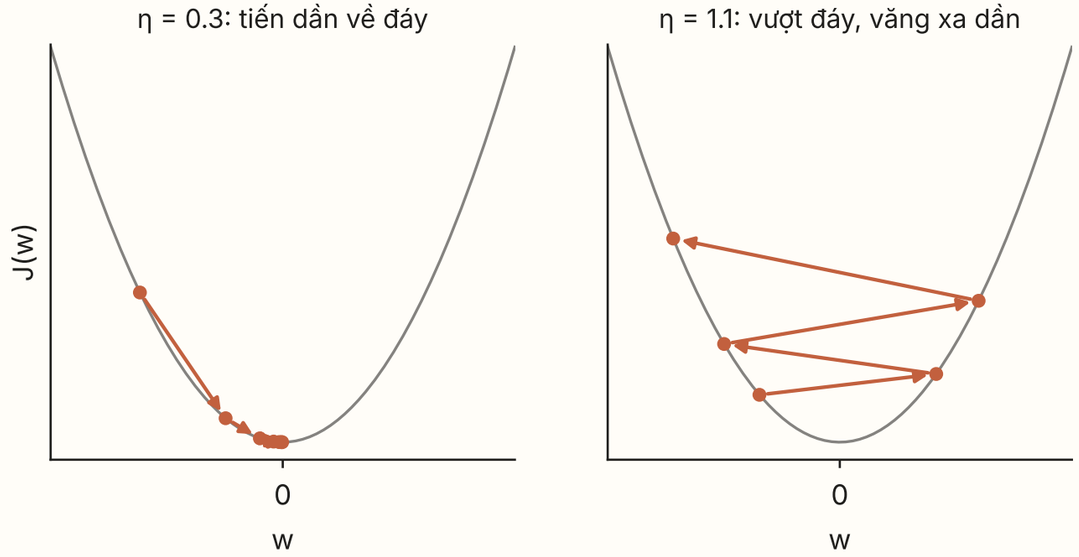
Có đoán trước được không?

Ngưỡng của learning rate

Hessian của J là hằng số $H = \frac{2}{n}\tilde{X}^\top \tilde{X}$. Mỗi bước nhân sai số theo hướng ứng với trị riêng λ với $(1 - \eta\lambda)$, nên cần

$$\eta < \frac{2}{\lambda_{\max}}$$

Dữ liệu căn hộ: ngưỡng $= 1.43 \cdot 10^{-4}$, số điều kiện $\kappa = \lambda_{\max}/\lambda_{\min} \approx 10^5$.



Bước vừa phải thì tiến về đáy; bước quá dài thì vượt đáy và văng xa dần.

HỎI NHANH

Loss in ra qua các vòng: `15.9, 41.2, 260, 1.7e4, ...` rồi thành nan. Nên làm gì trước tiên?

A Tăng số vòng lặp

B Tăng learning rate

C Giảm learning rate, hoặc chuẩn hóa đặc trưng

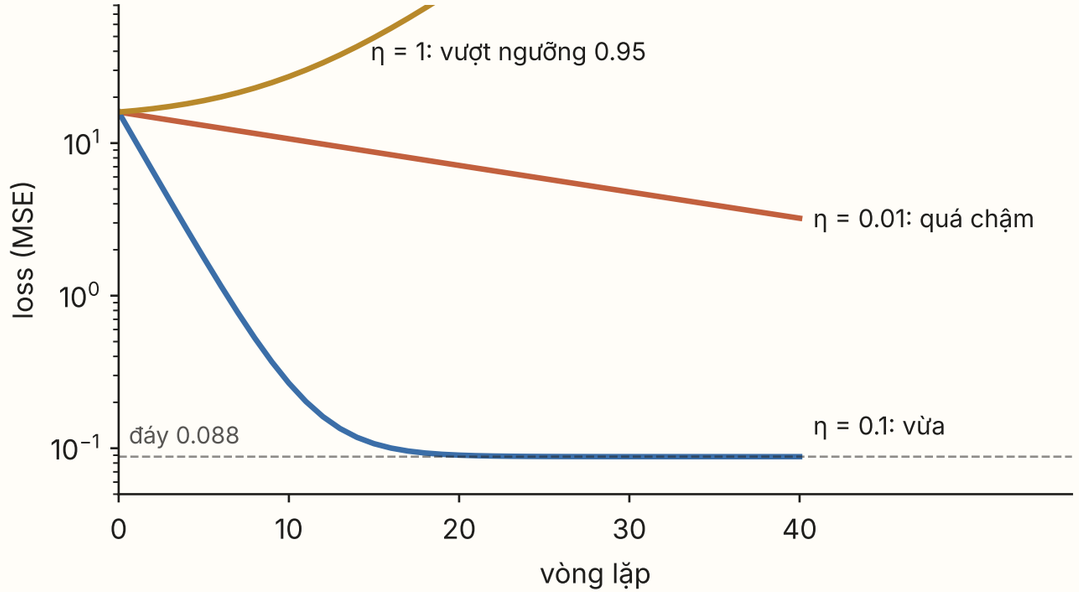
D Đổi seed ngẫu nhiên

Chuẩn hóa đặc trưng

$$z_j = \frac{x_j - \mu_j}{\sigma_j}$$

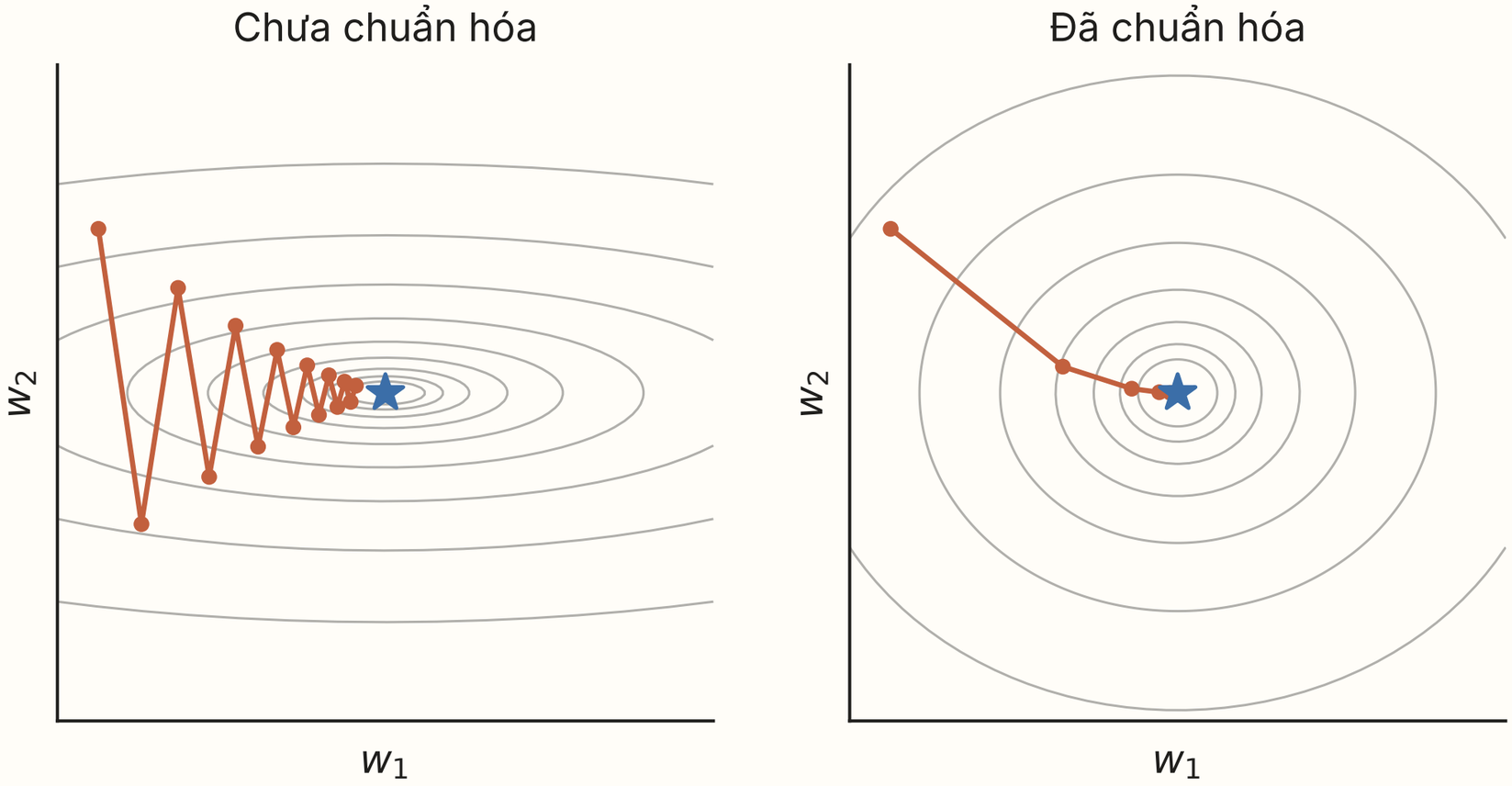
	Dữ liệu gốc	Đã chuẩn hóa
Số điều kiện	10^5	1.1
Ngưỡng η	$1.4 \cdot 10^{-4}$	0.95
Số vòng tới sát đáy	~180 000	28

μ, σ tính trên tập train, dùng lại cho test.



Loss theo từng vòng trên dữ liệu đã chuẩn hóa.

Thung lũng hẹp thành cái bát tròn



Đường đi của gradient descent trước và sau khi chuẩn hóa.

06

Thực hành và đọc kết quả

Dùng scikit-learn

```
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=0)
model = LinearRegression().fit(X_tr, y_tr)
print(model.score(X_tr, y_tr),
      model.score(X_te, y_te))
```

KẾT QUẢ

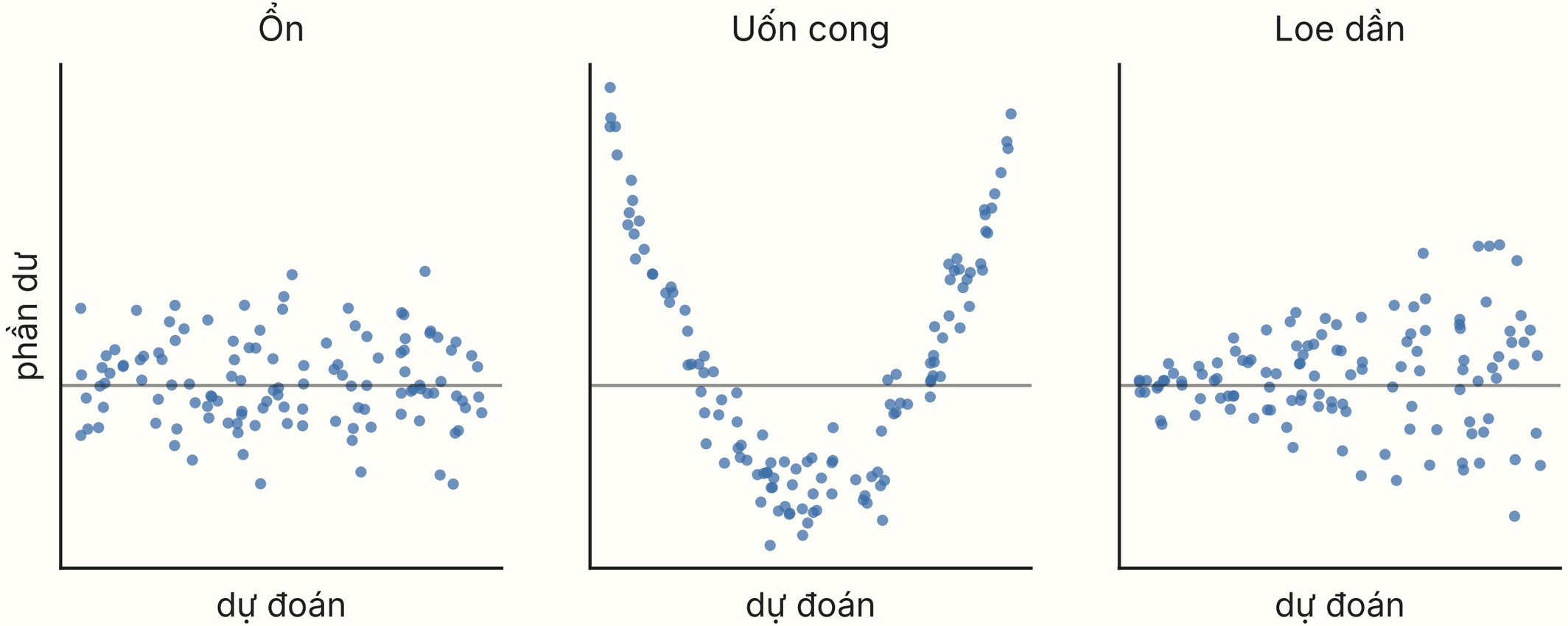
```
0.9604123189943734 0.9689505848749018
```

Kết quả trùng với code tự viết: `np.allclose` cho `True`.

Đọc hệ số cho đúng:

- Hệ số phụ thuộc đơn vị; muốn so sánh thì chuẩn hóa trước.
- Hệ số là mối liên hệ khi giữ các biến khác cố định, **không phải nhân quả**.
- Luôn vẽ phần dư theo dự đoán.

Đồ thị phần dư



Ổn: rải đều quanh 0. Uốn cong: thiếu đặc trưng phi tuyến. Loe dần: nên biến đổi y.

Bẫy: broadcasting âm thầm

```
y_col = y.reshape(-1, 1)          # (200, 1)
y_hat = predict(X, w_cf, b_cf)    # (200,)
print((y_hat - y_col).shape)
print(mse(y_hat, y), mse(y_hat, y_col))
```

KẾT QUẢ

```
(200, 200)
0.08807615180405967 4.594878280120061
```

NumPy kéo cả hai thành `(200, 200)`: không báo lỗi, loss trông hợp lý nhưng vô nghĩa.

- Đọc nhãn bằng `df["price"]`, không dùng `df[["price"]]`.
- `assert y_hat.shape == y.shape` ở đầu hàm loss.

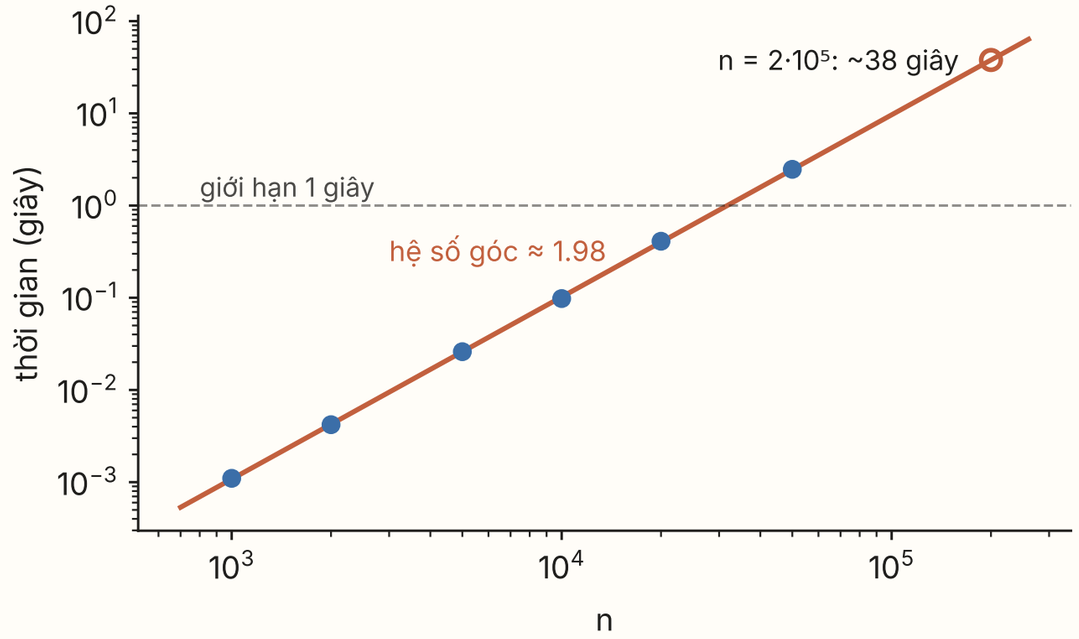
Đoán độ phức tạp trước khi nộp

"Tuyến tính" là theo tham số; đặc trưng có thể là x^2, $\log x$, $x_1 x_2$.

Nếu $t \approx C n^k$ thì

$$\log t = k \log n + \log C$$

Hồi quy trên trục log–log: hệ số góc ≈ 1.98, tức $O(n^2)$. Với $n = 2 \cdot 10^5$: khoảng 38 giây.



Thời gian chạy theo n trên trục log–log.

Khi đi thi

- **Luôn có baseline**: chạy `LinearRegression` trước tiên, làm mốc cho mọi mô hình sau.
- **Loss nổ**: chuẩn hóa đặc trưng trước, chỉnh learning rate sau.
- **Target lệch**: học trên $\log(1+y)$; gần như bắt buộc với RMSLE.
- **Ngoại suy**: mô hình tuyến tính kéo xu hướng ra ngoài vùng dữ liệu, mô hình cây thì không.
- **Tự viết thì tự kiểm**: kiểm shape và gradient trên dữ liệu nhỏ trước khi chạy thật.

Tóm tắt

- $\hat{y} = \mathbf{w}^\top \mathbf{x} + b$, học bằng cách cực tiểu MSE; R^2 so với mốc đoán trung bình.
- Nghiệm đóng: giải $\tilde{X}^\top \tilde{X} \boldsymbol{\theta} = \tilde{X}^\top \mathbf{y}$ bằng `solve`; về hình học là phép chiếu vuông góc.
- Gradient descent hội tụ khi $\eta < 2/\lambda_{\max}$; chuẩn hóa đặc trưng kéo số điều kiện về gần 1.
- Dùng scikit-learn với train/test; hệ số là liên hệ, không phải nhân quả.

Sau giờ học

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy notebook của bài: đổi learning rate, thử bỏ bước chuẩn hóa, và đọc lại kết quả.